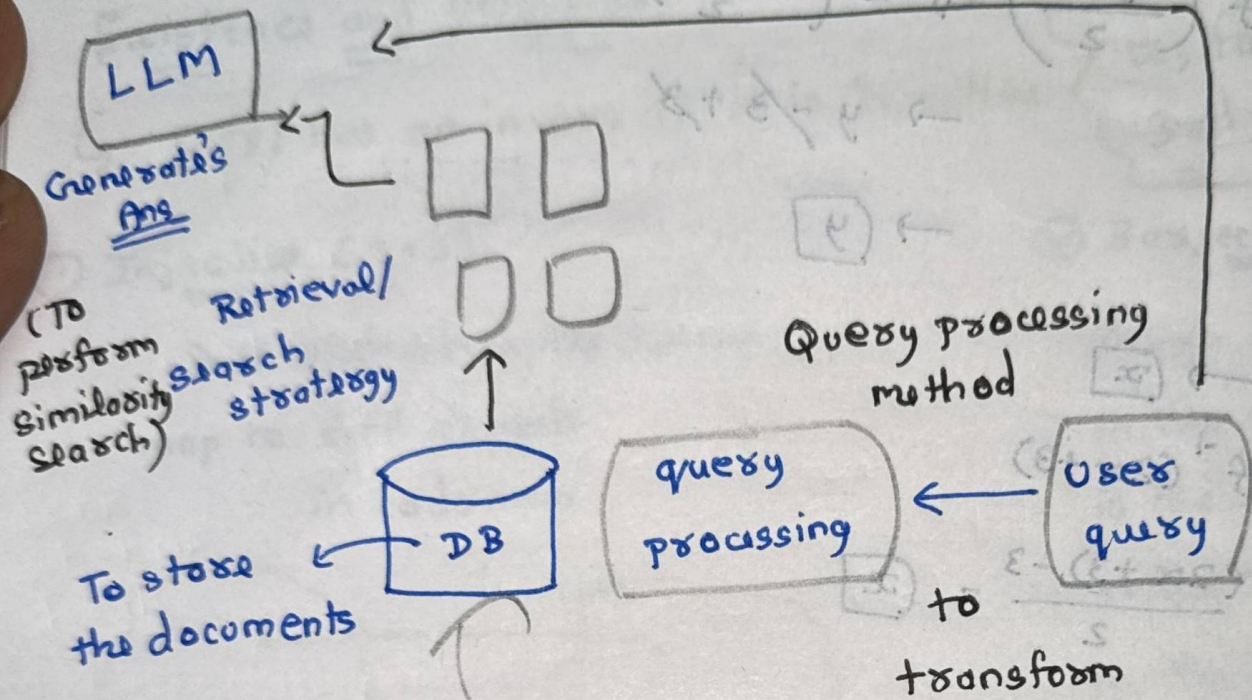


RAG Pipelines

Architecture

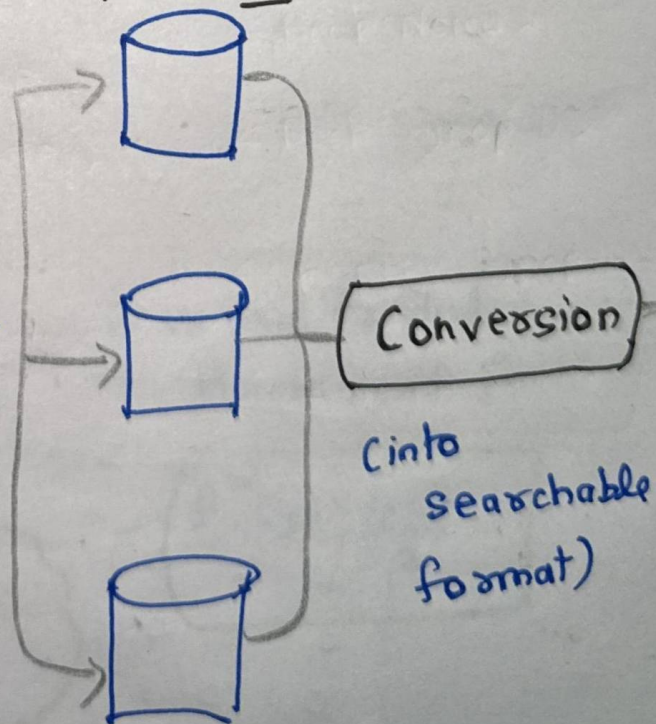


Pre-processing
Data (before deploying to the server)

find the
Data source
(wiki articles/
any other info
websites)

chunking
(break down
data to smaller
docs/chunks)

(insert them
into the DB)



NOTE

↳ RAGs uses

google search engine APIs

(in order to search for their
document)

Chunking

↳ Breaking out large

docs into smaller

and more manageable
pieces

so imp
that it could

make/break the RAG pipe line

- ① Sentence level chunking (difficult to retrieve thro' any search meth)
- ② Para level chunking
- ③ Entire document

↳ could confuse
the LLMs

(Numerical
Representation)

Split the chunks
store the data by
having a parameter
for chunks)

↳ then insert
them into the
DB

Problem of Data conversion

Goal → Converting text chunks to a searchable format

Data Conversion Approaches

Embedding based methods

Lang model Embeddings

GOOD

- ① Capturing Semantic relationships
- ② handling syn
- ③ understanding context-dependent meanings

Cons

Embedding would fetch docs kind of resembles your query
(fail to exactly match them)

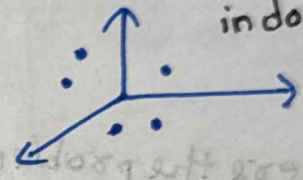
because of this we go for

keyword based retrieval

TF-IDF →
$$\frac{\text{No of times (t) appears in D}}{\text{total no. of terms in D}}$$

keyword frequency methods

TF-IDF, (Statistical Relationship b/w terms in docs & queries)
BM-25



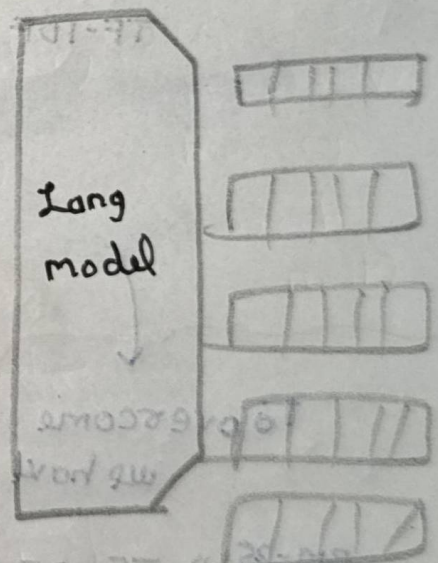
I am happy

positivity is king

sadness is life

Grief is me!

Blue Refrig



Every sentence
in the doc has
a TF-IDF score

Godrej A241gx coal	0.50	0.00	(Likewise they will have <u>scores</u>)
Food Fresh G A241	0.46	0.00	
Eco friendly 225521			
Powerful coal Godrej			
	A241gx	ab21	coal eco

Here the problem is

TF-IDF (good for the larger documents } even tho'
it is not
able to find
content there)

but the prob
of a word that
is appearing

To overcome
we have

So page > 5 page
does does

BM-25 $\rightarrow$ TF-IDF + length normalization + Term saturation

(if word appears
too often in DB
its imp (↓) see)

(Takes the of the
average of the length
of all documents
then align (as)
position the words)

Databases

Vector DBs (specialized for fast similarity check)

Pinecone

milvus

chroma

Drant

Mongo DB

Graph Databases

store info as n/w of DB's
with each doc connected
to others through
relationships

(Required when query
needs some
global
understanding
of concepts)

Query Rewriting / transformation

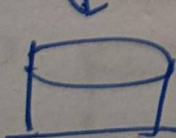
Direct Retrieval

Search

(before search
query rewriting
happens) using LLM

Leonardo da
Vinci mona
Lisa artist

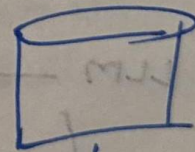
Search



Docs on
LPV

what happened
to the artist
who painted
the mona
Lisa

Search



Documents
on mona
Lisa

(here you won't
get proper info
on artist)
query not
modified

Sometimes

Contextual Query Writing

Translate queries if DB is a diff lang

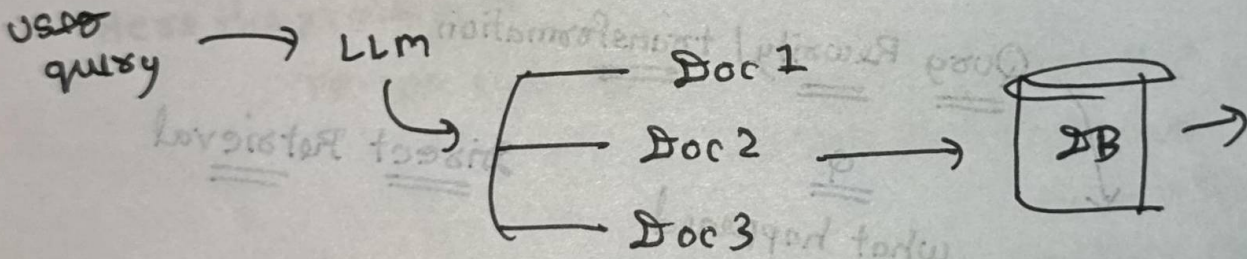
user query + App Level context = aware query

chat context

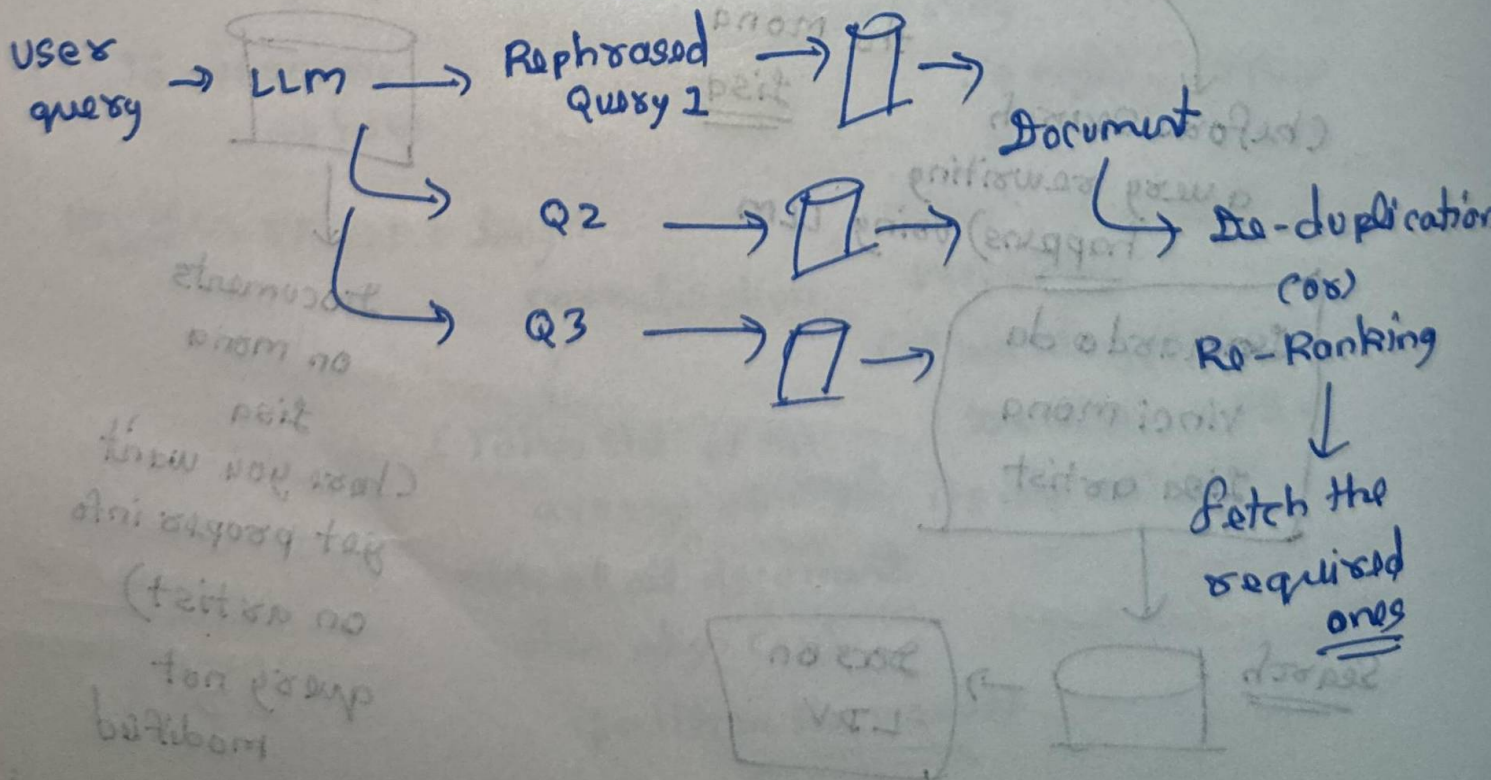
Additional Domain info

HYDE

↳ hypothetical Document Embeddings



Multi Query Expansion



Multi hop Retrieval

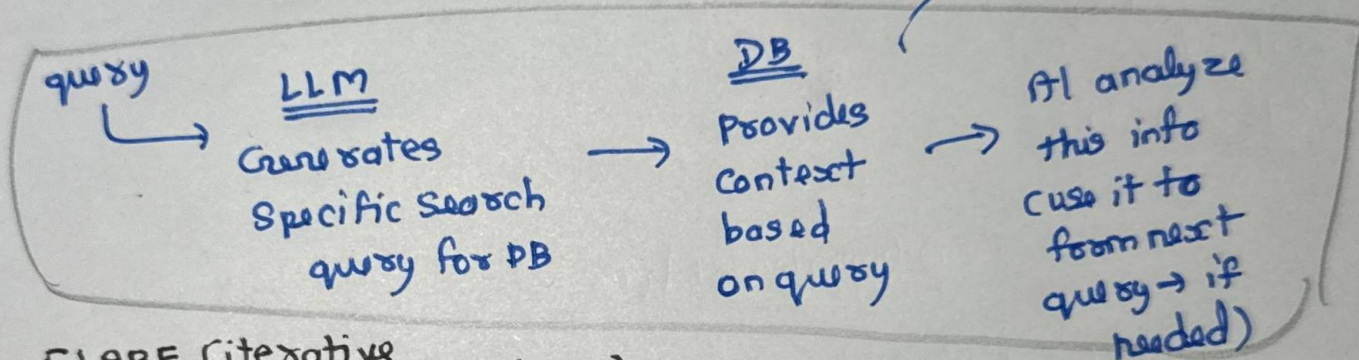
AI follows trail of clues
across multiple documents
(to ans complex ques)

System

↳ repeats a cycle of

- ① Searching
 - ② Learning
- } until it has enough info

gets like a loop



FLARE (iterative post Ret strategy)

LLM guesses temp next sentence

find low Prob tokens
↓ exist?

Retrieve new docs again

else
finish

loop ends

LLM combines all gathered facts into one response

(if loop ends)

Post Retrieval

Reciprocal Rank fusion

